

ICC-Tutorial: Demo and Exercises for Safety Verification of Neural Network-Controlled Cyber-Physical Systems

Lipsy Gupta¹, Ratan Lal², Pavithra Prabhakar³

¹Kansas State University

² Northwest Missouri State University

³ University of New Mexico

December 18, 2025

Here, we demonstrate **AVERINN** tool for some existing examples. Also, the users can create their own example and use the tool for the verification. We use the Google Colab platform to run the tool. Hence, the users does not need anything to install on their local computer.

1 Log in on the Google Colab

In this section, we provide steps to login on the Google Colab below.

1. Visit the Google Colab page via clicking on the link.
2. Create an account on the Google Colab via clicking on “**Sign in**” button on the top right corner, as shown in Figure 1. When the “**Sign in**” button is clicked, it will redirect to the **gmail** page, where users have to log in on their **gmail** account if it exists. Otherwise, they need to create a **gmail** account.

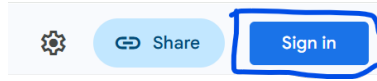


Figure 1: Sign in on Google Colab

3. After successful “**Sign in**” on the Google Colab, the page will look similar to the one shown in Figure 2.

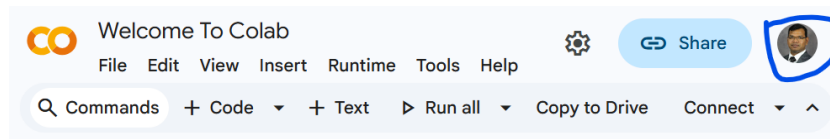


Figure 2: Log in on Google Colab

2 Upload a Jupyter Notebook on the Google Colab

In this section, we provide steps to upload a jupyter notebook on the Google Colab.

1. Download the Jupyter file named `icc.ipynb` given in the shared folder on your local computer.
2. Click on the **File** menu bar on the Google Colab and then click **upload notebook** option as shown in Figure 3.

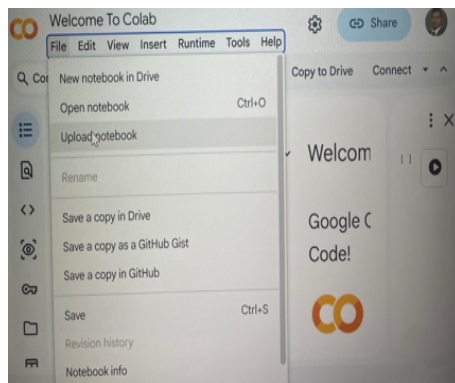


Figure 3: Upload notebook option in the Google Colab

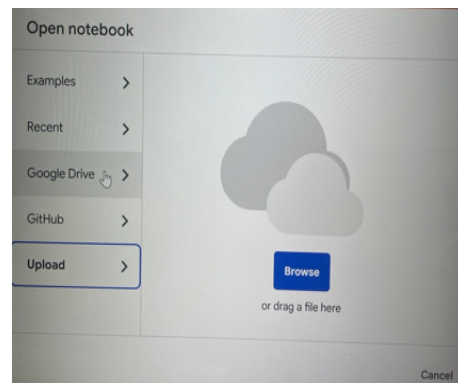


Figure 4: Brown option to Upload Notebook in the Google Colab

3. Click on **Browse** button as shown in Figure 4 and upload the downloaded `icc.ipynb` file.
4. Once the file is uploaded, click on **connect** button on the top right corner as shown in Figure 5.
5. Once the machine is connected, the **connect** button should be similar to the one shown in Figure 6.

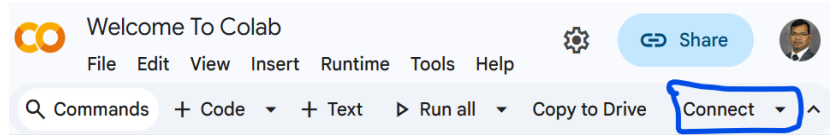


Figure 5: Connect a Server Machine on the Google Colab

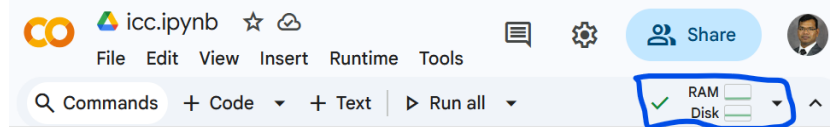


Figure 6: A Connected Machine on the Google Colab

3 Run AVERINN tool on the Google Colab

In this section, we provide steps to run the tool using the Jupyter notebook uploaded on the Google Colab.

1. After successfully uploading the Jupyter notebook file: `icc.ipynb` and connecting to a server machine on the Google Colab, please run all the code cells except the last four code cells.
2. Test Reachability/Safety of Neural Networks
 - The fourth cell from the last is related to reachability computation or safety checking of neural networks. During the execution, a log file named `log.txt` is created, where network information, specifications, and respective outputs are stored.
 - The third cell from the last is to print the contents of the log file.
3. Test Reachability/Safety of Neural Network Control Systems
 - The second cell from the last is related to reachability computation or safety checking of neural network-controlled systems. During the execution, a log file named `log.txt` is created, where network information, specifications, and respective outputs are stored.
 - The last cell is to print the contents of the log file.

4 Configuration for Neural Networks

In this section, we describe each part of the configuration that is required to be specified in the `nn-config.ini` file located in the `AVERINN` folder. Contents of the file `nn-config.ini` are shown in Listing 1. Since neural networks are expressed in different formats, the `AVERINN` tool supports them, namely, formats

used in SHERLOCK, ONNX, and NNET. It also supports interval neural networks expressed in SHERLOCK format, named ISHERLOCK. Next, we explain each line given in Listing 1. In line 4, `nnformat` captures a type of neural network file. In line 6, `nnpath` captures the path of a neural network file while `specpath` captures the path of a specification file in line 8. In line 10, `probType` is used to specify the type of problems, that is, REACH or SAFETY. The AVERINN tool provides interval abstraction of a neural network. If users want to abstract the neural network, they can specify `absRequired` to be “YES” otherwise “NO” in line 12. If `absRequired` is “YES”, then the number of abstract nodes at each hidden layer must be assigned to `numOfAbsNodes` in line 15. Further, the tool provides two methods, namely Mixed Integer Linear Programming (MILP) and Propagation-based, to solve both problems REACH and SAFETY. To specify a particular method to be used, `technique` has to be assigned with either MILP or PROPAGATION in line 17. Finally, in line 20, `lastRelu` captures whether the Relu activation function is applied at the last layer.

```

1 [settings]
2 # Neural Network format
3 #(SHERLOCK/ONNX/NNET/ISHERLOCK)
4 nnformat = "SHERLOCK"
5 # Path for a neural network file
6 nnpath = "./resources/test-nn/example1.txt"
7 # Path for a specification in vnnlib format
8 specpath = "./resources/test-nn/example1.vnnlib"
9 # Problem type REACH or SAFETY
10 probType = "SAFETY"
11 # YES if want to abstract, NO otherwise
12 absRequired = "YES"
13 # Number of abstract nodes at each hidden layer
14 # if absRequired is YES, otherwise ignore
15 numOfAbsNodes = 2
16 # Method type MILP/PROPAGATION
17 technique = "MILP"
18 # YES for relu on the last layer,
19 # NO for not applying relu on the last layer
20 lastRelu = "NO"
21 #####
22 # Following specification does not change #
23 #####
24 # Abstraction type
25 absType = "INTERVAL"
26 # Partition type
27 partitionType = "FIXED"
28 # Solver used for methods
29 solverType = "Gurobi"

```

Listing 1: Configuration for Neural Networks

5 Configuration of Neural Network Controlled Systems

In this section, we describe each part of the configuration that is required to be specified in the `nncs-config.ini` file located in the `AVERINN` folder.

```
1 [settings]
2 # Neural Network format
3 #(SHERLOCK/ONNX/NNET/ISHERLOCK)
4 nnformat = "SHERLOCK"
5 # Path for a neural network file
6 nnp_path = "./resources/test-nncs/example1.txt"
7 # Path for a specification in vnnlib format
8 spec_path = "./resources/test-nncs/example1.vnnlib"
9 # Problem type REACH or SAFETY
10 probType = "SAFETY"
11 # YES if want to abstract, NO otherwise
12 absRequired = "YES"
13 # Number of abstract nodes at each hidden layer
14 # if absRequired is YES, otherwise ignore
15 numOfAbsNodes = 2
16 # Method type MILP/PROPAGATION
17 technique = "PROPAGATION"
18 # YES for relu on the last layer,
19 # NO for not applying relu on the last layer
20 lastRelu = "NO"
21 # Dimension of the dynamical system
22 #  $X' = AX/AX+C/AX+BU/AX+BU+C$ 
23 sysDim = 2
24 inputDim = 1
25 # Matrix A for linear discrete time system  $AX+BU+C$ 
26 matA = [[1, 1], [1, -1]]
27 # Matrix B for linear discrete time system  $AX+BU+C$ 
28 # If B is not in the dynamics, use NONE
29 matB = [[0], [1]]
30 # Vector C for linear discrete time system  $AX+BU+C$ 
31 # If C is not in the dynamics, use NONE
32 arrayC = "NONE"
33 # Number of iteration (time steps)
34 K = 2
35 # Following specification does not change #
36 # Abstraction type
37 absType = "INTERVAL"
38 # Partition type
39 partitionType = "FIXED"
40 # Solver used for methods
41 solverType = "Gurobi"
```

Listing 2: Configuration for Neural Network Control Systems

Contents of the file `nncs-config.ini` are shown in Listing 2. In Listing 2, lines 1 to 20 are the same as in Listing 1. For a neural network-controlled system, we should specify the dynamics, which are given in lines 23, 24, 26, 29, and 32. Here, we perform bounded reachability or safety. So, we specify time steps by assigning a value to K .

6 Neural Network and Specification File

For the tutorial, we have used the SHERLOCK file format for specifying a neural network.

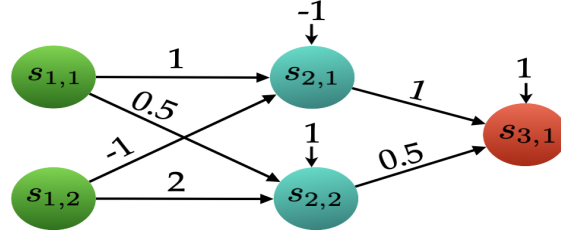


Figure 7: Neural Network

```

1 # Number of inputs
2 2
3 # Number of outputs
4 1
5 # Number of hidden layers
6 1
7 # Number of neurons at the first hidden layer
8 2
9 # weight from s_{1,1} to s_{0,1}
10 1
11 # weight s_{1,1} to s_{0,2}
12 -1
13 # Bias for neuron s_{1,1}
14 -1
15 # weight from s_{1,2} to s_{0,1}
16 0.5
17 # weight from s_{1,2} to s_{0,2}
18 2
19 # Bias for neuron s_{1,2}
20 1
21 # weight from s_{2,1} to s_{1,1}
22 1
23 # weight from s_{2,1} to s_{1,2}
24 0.5
25 # Bias for neuron s_{2,1}
26 1

```

Listing 3: Neural Network in SHERLOCK

The details about the format can be found via clicking on link. For the specification file, we have used VNNLIB format for an input set and unsafe specification, which is similar to SMTLIB format. The details about the SMTLIB can be found via clicking on link. Next, we demonstrate with an example. Let us consider the neural network shown in Figure 7.

Next, we specify an input set as a Box, where both input neurons will be in an interval $[1,2]$, that is, $[1,2] \times [1,2]$, and the unsafe specification is that the value of the output neuron is greater than 10. Both can be expressed in a VNNLIB file as shown in Listing 4.

```

1 ; Input Variables
2 (declare-const X_0 Real)
3 (declare-const X_1 Real)
4 ; Output Variables
5 (declare-const Y_0 Real)
6 ; Constraints for input set
7 (assert (<= X_0 2))
8 (assert (>= X_0 1))
9 (assert (<= X_1 2))
10 (assert (>= X_1 1))
11 ; Constraints for unsafe
12 (assert (>= Y_0 10))

```

Listing 4: Specification

7 Demo for Neural Networks

In this section, users perform reachability/safety for a neural network specified in `resources/test-nn` folder. For each specified network, there is a corresponding specification file in the same folder. For example, for a neural network specified in `example1.txt` file, its specification file is `example1.vnnlib`.

- **Reachability:**

1. open `nn-config.ini` file available within AVERINN folder, and ensure that the `probType` is “REACH” as shown in Listing 6.

```

1 [settings]
2 # Problem type REACH or SAFETY
3 probType = "REACH"

```

Listing 5: Reachability Configuration

2. Reachability can be analyzed by two methods, namely, MILP and PROPAGATION. Here, we analyze using MILP. Users can analyze the same using PROPAGATION method later.

```

1 [settings]
2 # Method type MILP/PROPAGATION
3 technique = "MILP"

```

Listing 6: Technique Configuration

3. Initial set is considered as a Box, that is, $[1, 2] \times [1, 2]$. In order to change the initial set, the user must open `example1.vnnlib` file, where input range can be changed.
 4. Reachability can be analyzed via executing appropriate cell in the Google Colab. Then, its output can be seen via executing the cell for printing log file in the Google Colab.
- **Safety:** Similarly, safety can be analyzed via modifying `probType` to `SAFETY` in `nn-config.ini` file. In order to change unsafe specification, users must pen `example1.vnnlib` file, where unsafe constraints on output neurons can be changed. Rest of the steps are the same.
 - **Other Examples:** Similarly, other examples available in `resources/test-nn` folder can be analyzed for both reachability and safety. In order to test other examples, users must change `nnpath` and `specpath` in the file `nn-config.ini` as shown in Listing 7.

```

1 [settings]
2 # Path for a neural network file
3 nnpath = "../resources/test-nn/example1.txt"
4 # Path for a specification in vnnlib format
5 specpath = "../resources/test-nn/example1.vnnlib"

```

Listing 7: Path Configuration

8 Exercises for Neural Networks

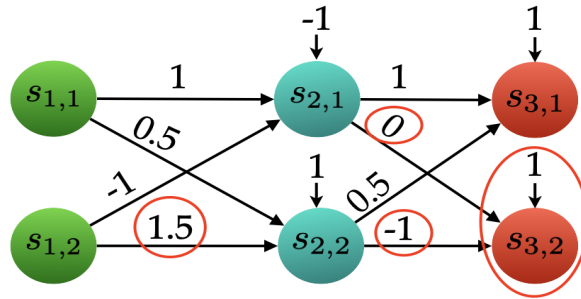


Figure 8: Neural Network

- Q1: Modify the `example1.txt` file located in the `resources/test-nn` folder to specify the neural network shown in Figure 8, with the differences from the network in Figure 7 clearly highlighted.

- Q2: Modify the `example1.vnnlib` file located in the `resources/test-nn` folder to declare an additional variable corresponding to the new output neuron shown in Figure 8, and add a constraint enforcing that this new variable is greater than or equal to 10.
- Q3: Compute reach set for the network shown in Figure 8 using the MILP technique. Reach set should be the following:

Lower Bound : $[2.5, -4.0]$ Upper Bound : $[3.5, -2.0]$

[Hint: Do appropriate modification in the `nn-config.ini` if needed.]

- Q4: Compute reach set for the network shown in Figure 8 using the PROPAGATION technique. Reach set should be the following:

Lower Bound : $[2.5, -4.0]$ Upper Bound : $[3.5, -2.0]$

[Hint: Do appropriate modification in the `nn-config.ini` if needed.]

- Q5: Verify safety of the network shown in Figure 8 against unsafe specification given in the modified version of `example1.vnnlib` using the MILP technique. Safety status should be **Safe**. [Hint: Do appropriate modification in the `nn-config.ini` if needed.]

9 Demo for Neural Network Controlled Systems

In this section, users perform reachability/safety for a neural network controlled system specified in `resources/test-nnCS` folder. For each specified network, there is a corresponding specification file in the same folder. For example, for a neural network specified in `example1.txt` file, its specification file is `example1.vnnlib`.

- **Reachability:**

1. open `nnCS-config.ini` file available within `AVERINN` folder, and ensure that the `probType` is “REACH” as shown in Listing 6.

```
1 [settings]
2 # Problem type REACH or SAFETY
3 probType = "REACH"
```

Listing 8: Reachability Configuration

2. Reachability can be analyzed by two methods, namely, MILP and PROPAGATION. Here, we analyze using MILP. Users can analyze the same using PROPAGATION method later.

```
1 [settings]
2 # Method type MILP/PROPAGATION
3 technique = "MILP"
```

Listing 9: Technique Configuration

3. For changing dynamics of the system, users can modify the following in `nnCS-config.ini` file while ensuring number of state and input variables are aligned with the neural network used for the controller.

```
1 [settings]
2 # Dimension of the dynamical system
3 # X' = AX/AX+C/AX+BU/AX+BU+C
4 sysDim = 2
5 inputDim = 1
6 # Matrix A for linear discrete time system AX+BU+
  C
7 matA = [[1, 1], [1, -1]]
8 # Matrix B for linear discrete time system AX+BU+
  C
9 # If B is not in the dynamics, use NONE
10 matB = [[0], [1]]
11 # Vector C for linear discrete time system AX+BU+
  C
12 # If C is not in the dynamics, use NONE
13 arrayC = "NONE"
14 # Number of iteration (time steps)
15 K = 2
```

Listing 10: Configuration for Dynamics

4. Initial set is considered as a Box, that is, $[1, 2] \times [1, 2]$. In order to change the initial set, the user must open `example1.vnnlib` file, where input range can be changed.
 5. Reachability can be analyzed via executing appropriate cell in the Google Colab. Then, its output can be seen via executing the cell for printing log file in the Google Colab.
- **Safety:** Similarly, safety can be analyzed via modifying `probType` to `SAFETY` in `nncs-config.ini` file. In order to change unsafe specification, users must pen `example1.vnnlib` file, where unsafe constraints on output neurons can be changed. Rest of the steps are the same.
 - **Other Examples:** Similarly, other examples available in `resources/test-nncs` folder can be analyzed for both reachability and safety. In order to test other examples, users must change `nnpPath` and `specPath` in the file `nncs-config.ini` as shown in Listing 11.

```

1 [settings]
2 # Path for a neural network file
3 nnpPath = "./resources/test-nncs/example1.txt"
4 # Path for a specification in vnnlib format
5 specPath = "./resources/test-nncs/example1.vnnlib"

```

Listing 11: Path Configuration

10 Exercises for Neural Network Control Systems

Here, we have retained the solutions to Q1 and Q2 from Section 8 in the files `example1P.txt` and `example1P.vnnlib`, located in the `resources/test-nncs` folder.

- Q1: Consider the following dynamics:

$$\begin{aligned}
 x_1(t+1) &= x_1(t) + 2 * x_2(t) + u_1(t) \\
 x_2(t+1) &= 2 * x_1(t) - x_2(t) + u_2(t)
 \end{aligned}$$

Modify `nncs-config.ini` file for the above dynamics, that is, determine `matA` and `matB`.

- Q2: Compute reach set for the network network controlled system with dynamics given above and neural network shown in Figure 8 for 2 time steps using the MILP technique. Reach set should be the following:

$$\text{Lower Bound : } [6.875, 9.25] \quad \text{Upper Bound : } [22.625, 20.0]$$

- Q3: Compute reach set for the network network controlled system with dynamics given above and neural network shown in Figure 8 for 2 time steps using the **PROPAGATION** technique. Reach set should be the following:

Lower Bound : $[4.625, 3.75]$ Upper Bound : $[25.0, 24.0]$

- Q4: Verify safety for the network network controlled system with dynamics given above and neural network shown in Figure 8 for 2 time steps using the **MILP** technique. Safety status should be **Unsafe**.